

PROYECTO AW · ISPTEC

Objetivo: 20 / 20 · Epoca Normal

Como Ler Este Plano — Guia Rápido

Plano Definitivo e Estrutural do Sistema

Este documento é grande (mais de 1500 linhas), por isso aqui está a forma mais inteligente de o ler consoante quem és:

👉 Se és **BE-Dev (Backend / Willfredy)**

Lê na seguinte ordem:

1. **Secções 1, 2, 3** (entender o projeto e divisão).
2. **Secção 5** ("BACKEND — O Que Construir") — toda. **É o teu manual de instruções.**
3. **Secção 6** (contrato API) — **decora**. Se mudares aqui, partes o frontend.
4. **Secção 7** (Git/GitHub).
5. **Secção 11** ("Roadmap de Estudos Avançados") — para o exame final.

👉 Se és **FE-Dev (Emer, Jeovani ou Manuel)**

Lê na seguinte ordem:

1. **Secções 1, 2, 3** (entender o projeto e divisão).
2. **Secção 4** ("FRONTEND — Angular") — toda. **É o vosso manual de instruções.**
3. **Secção 6** (contrato API) — **decora**. É o "contrato" que o backend te entrega; tudo o que o teu Angular chama está aqui.
4. **Secção 7** (Git/GitHub).
5. **Plano da 2.ª Parcelar — Secção 10** — o teu roadmap de estudos.

👉 Para o grupo inteiro (primeira reunião)

1. Combinar o **nome do repositório GitHub** (sugestão: `nzolanet`).
2. **Convidar todos** como colaboradores (Secção 7.2).
3. **Definir a estrutura de pastas** (`/backend` , `/frontend` , `/docs` — Secção 7.1).
4. **Aceitar o contrato API** (Secção 6) como fonte da verdade.
5. **Distribuir os módulos** do frontend entre os 3 FE-Devs (Secção 3.1).

💡 **Conselho:** este plano é a vossa "bíblia" durante todo o semestre. Mantenham-no aberto enquanto trabalham. Se algo mudar, atualizem aqui e comuniquem no grupo.

Índice

1. Análise do Enunciado
 2. Visão Geral da Arquitetura
 3. Divisão da Equipa
 4. FRONTEND — Angular (3 elementos)
 5. BACKEND — ASP.NET Web API + SQL Server (1 elemento)
 6. Contrato da API (Frontend ↔ Backend)
 7. Git e GitHub — Estratégia de Colaboração
 8. Cronograma de Entregas
 9. Checklist Final para 20/20
 10. Pressupostos Técnicos Assumidos
 11. Roadmap de Estudos Avançados (Pós-Parcelar)
 12. Glossário de Termos Técnicos
-

1. Análise do Enunciado

1.1. Domínios do sistema

Domínio	Funcionalidades-chave
Utilizadores	Registo, login, recuperação de senha, edição de perfil, foto de perfil, seguir/deixar de seguir, privacidade (público/privado)
Publicações	CRUD próprio, texto + imagem (opcional) + vídeo (opcional), visualização cronológica
Bazes	Dar/remover (toggle), contador, unicidade por utilizador/publicação
Comentários	CRUD próprio, listagem por publicação, moderação por administrador
Feed de Notícias	Publicações recentes + de utilizadores seguidos, ordem cronológica, atualização dinâmica
Notificações	Gera notificação ao receber base, comentário ou novo seguidor

1.2. Regras de negócio críticas

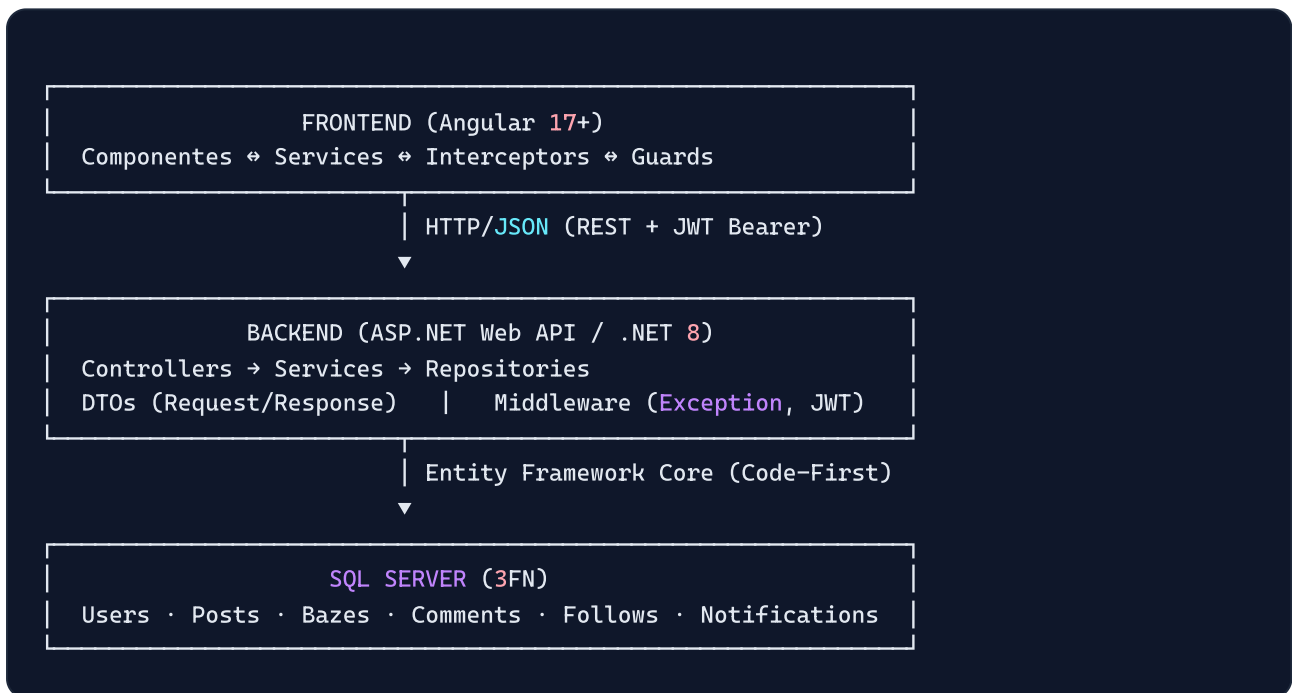
1. Apenas utilizadores **autenticados** podem publicar, comentar ou dar base.
2. Cada utilizador só pode **editar/excluir conteúdos da sua própria autoria**.
3. **1 base por utilizador por publicação** (garantido por constraint UNIQUE + lógica no serviço).
4. **Administrador** pode remover qualquer comentário considerado ofensivo.

5. **Perfis privados** só são visíveis para utilizadores autorizados (interpretado como: dono + seguidores).
6. **Notificações automáticas** ao receber base, comentário ou novo seguidor (nunca auto-notificar).
7. Uma publicação exibe: nome do autor, foto do autor, data, texto, imagem/vídeo opcional, contagem de bases e contagem de comentários.

1.3. Requisitos não-funcionais

- Interface responsiva (mobile-first).
- Segurança na autenticação (JWT).
- Proteção contra acessos não autorizados (Guards no Angular + `[Authorize]` na API).
- Boa performance (paginação, lazy loading, indexação na BD).
- Compatibilidade móvel e desktop.
- Usabilidade intuitiva.

2. Visão Geral da Arquitetura



Repositório único (monorepo):

```
nzolanet/  
├── backend/      ← ASP.NET Web API (Willfredy)  
├── frontend/    ← Angular (3 colegas)  
├── docs/        ← API_CONTRACT.md, ERD, relatório  
└── README.md
```

3. Divisão da Equipa

Elemento	Função	Responsabilidade principal
Emer Tavares (FE-Dev 1)	Frontend	Módulo <code>auth/</code> + guards + interceptors + edição de perfil
Jeovani Sassombo (FE-Dev 2)	Frontend	Módulo <code>feed/</code> + criação/edição de publicações + componente <code>post-card</code> + toggle de base
Manuel Sulo (FE-Dev 3)	Frontend	Módulo <code>notifications/</code> + módulo <code>comments/</code> + seguir/deixar de seguir + perfil (visualização)
BE-Dev (Willfredy)	Backend	Toda a Web API, SQL Server, autenticação JWT, regras de negócio, upload de ficheiros, relatório técnico

Regra de ouro: qualquer mudança ao **contrato API** (endpoints, DTOs) deve ser comunicada ao grupo e atualizada em `docs/API_CONTRACT.md` **antes** de ser implementada.

4. FRONTEND — Angular (3 elementos)

4.1. Tecnologias e bibliotecas

Tecnologia	Versão	Papel
Angular	17+ (standalone ou NgModules)	Framework principal
TypeScript	5+	Linguagem
Angular Router	built-in	Navegação + lazy loading
Angular HttpClient	built-in	Comunicação REST
Angular Reactive Forms	built-in	Formulários com validação
Angular Guards / Interceptors	built-in	Proteção de rotas + injeção de JWT
RxJS	built-in	Reatividade (BehaviorSubject, Observable)
TailwindCSS ou Bootstrap 5	latest	Estilização responsiva (escolher 1)
ngx-toastr	latest	Notificações visuais (toasts)
ngx-infinite-scroll	latest	Scroll infinito no feed
date-fns	latest	Formatação de datas ("há 2 horas")

Recomendação: TailwindCSS pela velocidade de prototipagem responsiva; alternativa Bootstrap 5 se a equipa dominar melhor.

4.2. Estrutura de pastas — Frontend

```
nzolanet-frontend/
├── src/
│   ├── app/
│   │   ├── core/                                # Singleton: carregado 1x, injetado globalmente
│   │   │   ├── guards/
│   │   │   │   ├── auth.guard.ts                # Bloqueia rotas sem JWT
│   │   │   │   ├── guest.guard.ts               # Redireciona logado p/ feed
│   │   │   │   └── admin.guard.ts               # Apenas admins
│   │   │   ├── interceptors/
│   │   │   │   ├── jwt.interceptor.ts           # Injeta "Bearer <token>" em todo pedido HTTP
│   │   │   │   └── error.interceptor.ts         # Trata 401/403/404 globalmente + toasts
│   │   │   ├── models/                          # Interfaces TypeScript (espelham DTOs do backen
│   │   │   │   ├── user.model.ts
│   │   │   │   ├── post.model.ts
│   │   │   │   ├── comment.model.ts
│   │   │   │   ├── baze.model.ts
│   │   │   │   ├── notification.model.ts
│   │   │   │   ├── auth.model.ts
│   │   │   │   └── paged-result.model.ts
│   │   │   ├── services/                        # Comunicação com a API REST
│   │   │   │   ├── auth.service.ts
│   │   │   │   ├── user.service.ts
│   │   │   │   ├── post.service.ts
│   │   │   │   ├── comment.service.ts
│   │   │   │   ├── baze.service.ts
│   │   │   │   ├── feed.service.ts
│   │   │   │   ├── notification.service.ts
│   │   │   │   └── upload.service.ts
│   │   │   └── constants/
│   │   │       └── api-endpoints.ts             # URLs centralizadas
│   │   ├── shared/                             # Reutilizável entre módulos
│   │   │   ├── components/
│   │   │   │   ├── navbar/
│   │   │   │   ├── sidebar/
│   │   │   │   ├── post-card/                  # Card de publicação reutilizável
│   │   │   │   ├── comment-item/
│   │   │   │   ├── comment-form/
│   │   │   │   ├── like-button/               # Toggle de baze com optimistic update
│   │   │   │   ├── user-avatar/               # Avatar com fallback
│   │   │   │   ├── notification-bell/         # Sino com contador
│   │   │   │   ├── media-preview/             # Pré-visualização imagem/vídeo
│   │   │   │   ├── confirm-dialog/
│   │   │   │   ├── loading-spinner/
│   │   │   │   └── empty-state/
│   │   │   ├── directives/
│   │   │       └── click-outside.directive.ts
│   │   └── pipes/
│   │       └── time-ago.pipe.ts                # "há 2 horas"
```

```

└─ safe-url.pipe.ts      # Sanitização de URLs (vídeos)

└─ layouts/
  └─ public-layout/      # Login, registo, recover
  └─ private-layout/     # Navbar + sidebar + conteúdo

└─ features/             # Módulos lazy-loaded por domínio
  └─ auth/               ← [Emer]
    └─ login/
    └─ register/
    └─ forgot-password/
    └─ reset-password/
    └─ auth.module.ts

  └─ feed/               ← [Jeovani]
    └─ feed-page/        # Lista cronológica + infinite scroll
    └─ create-post/      # Formulário com upload media
    └─ feed.module.ts

  └─ posts/              ← [Jeovani]
    └─ post-detail/
    └─ edit-post/
    └─ posts.module.ts

  └─ profile/            ← [Emer + Manuel]
    └─ profile-page/     # Visualização (Manuel)
    └─ edit-profile/     # Edição + foto (Emer)
    └─ followers-list/
    └─ profile.module.ts

  └─ comments/           ← [Manuel]
    └─ comment-list/
    └─ comments.module.ts

  └─ notifications/      ← [Manuel]
    └─ notifications-page/
    └─ notifications.module.ts

  └─ admin/              ← (opcional) [Manuel]
    └─ comment-moderation/
    └─ admin.module.ts

└─ app.component.ts
└─ app.module.ts
└─ app-routing.module.ts

└─ assets/
  └─ images/
  └─ icons/

└─ environments/
  └─ environment.ts      # apiUrl: 'http://localhost:5000/api'
  └─ environment.prod.ts

└─ styles/
  └─ styles.scss

```

```
|      └─ _variables.scss
|─ proxy.conf.json           # Proxy /api → backend local
|─ angular.json
|─ tsconfig.json
|─ package.json
```

4.3. Modelos TypeScript (espelham os DTOs do backend)

TYPESCRIPT

```
// core/models/user.model.ts
export interface User {
  id: number;
  username: string;
  email: string;
  fullName: string;
  bio?: string;
  profilePhotoUrl?: string;
  isPrivate: boolean;
  isAdmin: boolean;
}

export interface UserProfile extends User {
  followersCount: number;
  followingCount: number;
  postsCount: number;
  isFollowing: boolean; // contexto do utilizador autenticado
}

// core/models/post.model.ts
export interface Post {
  id: number;
  authorId: number;
  authorName: string;
  authorUsername: string;
  authorPhotoUrl?: string;
  content: string;
  imageUrl?: string;
  videoUrl?: string;
  bazesCount: number;
  commentsCount: number;
  userHasBazed: boolean; // para toggle visual otimista
  createdAt: string;
  updatedAt: string;
}

// core/models/comment.model.ts
export interface Comment {
  id: number;
  postId: number;
  authorId: number;
  authorName: string;
  authorPhotoUrl?: string;
  content: string;
  isEditable: boolean; // true se o autor é o utilizador atual
  createdAt: string;
}

// core/models/notification.model.ts
export interface Notification {
```

```

    id: number;
    type: 'Base' | 'Comment' | 'Follow';
    fromUserId: number;
    fromUsername: string;
    fromPhotoUrl?: string;
    postId?: number;
    isRead: boolean;
    createdAt: string;
  }

// core/models/auth.model.ts
export interface LoginRequest { email: string; password: string; }
export interface RegisterRequest {
  username: string; email: string; password: string; fullName: string;
}
export interface AuthResponse { token: string; expiration: string; user: User; }

// core/models/paged-result.model.ts
export interface PagedResult<T> {
  items: T[];
  page: number;
  pageSize: number;
  totalCount: number;
  totalPages: number;
}

```

4.4. Services principais

TYPESCRIPT

```
// core/services/auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService {
  private apiUrl = environment.apiUrl;
  currentUser$ = new BehaviorSubject<User | null>(null);

  login(data: LoginRequest): Observable<AuthResponse> { /* POST /auth/login */ }
  register(data: RegisterRequest): Observable<AuthResponse> { /* POST /auth/register */ }
  forgotPassword(email: string): Observable<void> { /* POST /auth/forgot-password */ }
  resetPassword(token: string, newPassword: string): Observable<void> { }
  logout(): void { localStorage.removeItem('token'); this.currentUser$.next(null); }
  getToken(): string | null { return localStorage.getItem('token'); }
  isAuthenticated(): boolean { return !!this.getToken(); }
  getCurrentUserId(): number | null { /* decodifica JWT */ }
}

// core/services/post.service.ts
@Injectable({ providedIn: 'root' })
export class PostService {
  createPost(formData: FormData): Observable<Post> { /* POST /posts */ }
  updatePost(id: number, dto: UpdatePostDto): Observable<Post> { }
  deletePost(id: number): Observable<void> { }
  getById(id: number): Observable<Post> { }
  getByUser(userId: number, page: number): Observable<PagedResult<Post>> { }
}

// core/services/feed.service.ts
@Injectable({ providedIn: 'root' })
export class FeedService {
  getFeed(page: number, pageSize: number): Observable<PagedResult<Post>> {
    /* GET /feed?page=&pageSize= */
  }
}

// core/services/baze.service.ts
@Injectable({ providedIn: 'root' })
export class BazeService {
  toggle(postId: number): Observable<{ bazesCount: number; userHasBazed: boolean }> {
    /* POST /posts/{id}/baze (idempotente) */
  }
}

// core/services/notification.service.ts
@Injectable({ providedIn: 'root' })
export class NotificationService {
  unreadCount$ = new BehaviorSubject<number>(0);

  getAll(page: number): Observable<PagedResult<Notification>> { }
  markAsRead(id: number): Observable<void> { }
  markAllAsRead(): Observable<void> { }
```

```
startPolling(intervalMs = 30000): void { /* atualiza badge */ }  
}
```

4.5. Interceptors

TYPESCRIPT

```
// core/interceptors/jwt.interceptor.ts  
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {  
  const token = inject(AuthService).getToken();  
  if (token) req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });  
  return next(req);  
};  
  
// core/interceptors/error.interceptor.ts  
export const errorInterceptor: HttpInterceptorFn = (req, next) => {  
  const router = inject(Router);  
  const toastr = inject(ToastrService);  
  return next(req).pipe(catchError((err: HttpResponse) => {  
    switch (err.status) {  
      case 401: router.navigate(['/login']); break;  
      case 403: toastr.error('Sem permissão para esta ação.');
```

4.6. Rotas com Lazy Loading

TYPESCRIPT

```
// app-routing.module.ts
const routes: Routes = [
  { path: '', redirectTo: 'feed', pathMatch: 'full' },

  // Rotas públicas
  {
    path: '',
    component: PublicLayoutComponent,
    canActivate: [guestGuard],
    children: [
      { path: 'auth', loadChildren: () => import('./features/auth/auth.module').then(m => m)
    ]
  },

  // Rotas privadas
  {
    path: '',
    component: PrivateLayoutComponent,
    canActivate: [authGuard],
    children: [
      { path: 'feed', loadChildren: () => import('./features/feed/feed.module').th
      { path: 'posts', loadChildren: () => import('./features/posts/posts.module').
      { path: 'profile/:id', loadChildren: () => import('./features/profile/profile.modul
      { path: 'notifications', loadChildren: () => import('./features/notifications/notific
      { path: 'admin', canActivate: [adminGuard],
        loadChildren: () => import('./features/admin/admin.module').
    ]
  },

  { path: '**', redirectTo: 'feed' }
];
```

4.7. Distribuição de trabalho — Frontend (3 devs)

Dev	Branch Git	Módulos / Componentes
Emer Tavares (FE-Dev 1)	<code>frontend/feature/auth</code>	<code>auth/</code> (login, register, forgot/reset), <code>auth.guard</code> , <code>guest.guard</code> , <code>jwt.interceptor</code> , <code>error.interceptor</code> , <code>AuthService</code> , <code>edit-profile/</code>
Jeovani Sassombo (FE-Dev 2)	<code>frontend/feature/feed</code>	<code>feed/</code> (feed-page + infinite scroll), <code>create-post/</code> , <code>posts/edit-post</code> , <code>post-card</code> (shared), <code>like-button</code> (shared), <code>media-preview</code> (shared), <code>PostService</code> , <code>FeedService</code> , <code>BazeService</code> , <code>UploadService</code>
Manuel Sulo (FE-Dev 3)	<code>frontend/feature/comments-notifications</code>	<code>comments/</code> , <code>notifications/</code> , <code>notification-bell</code> (shared), <code>profile-page/</code> , <code>followers-list/</code> , <code>admin/comment-moderation</code> , <code>CommentService</code> , <code>NotificationService</code> , <code>UserService</code> (follow/unfollow)

Componentes shared: quem criar é "dono"; alterações por outro dev passam **obrigatoriamente** por PR com revisão.

4.8. Regras de UI obrigatórias

- **Botões "Editar" e "Apagar"** só aparecem se `post.authorId === currentUserId` (idem para comentários).
- **Botão de baze:** actualização otimista da UI (incrementa contador antes da resposta) com rollback em caso de erro.
- **Feed:** ordem cronológica (`createdAt DESC`), scroll infinito ou paginação com botão "Carregar mais".
- **Perfil privado:** se o utilizador não segue, mostrar mensagem *"Este perfil é privado. Segue para ver as publicações."*
- **Upload:** pré-visualização da imagem/vídeo antes do submit, validação client-side de tipo (`.jpg`, `.png`, `.mp4`) e tamanho (≤ 10 MB para imagens, ≤ 50 MB para vídeos).
- **Responsividade:** testar em 320px (mobile), 768px (tablet), 1024px (desktop). Sidebar colapsa em menu hambúrguer ≤ 768px.
- **Feedback visual:** spinners durante chamadas HTTP, toasts em todos os sucessos/erros, skeleton loaders nas listas.

- **Acessibilidade:** `alt` em imagens, `aria-label` em botões sem texto, navegação por teclado.

4.9. Configuração de proxy (desenvolvimento)

JSON

```
// proxy.conf.json - evita problemas de CORS em dev
{
  "/api": {
    "target": "http://localhost:5000",
    "secure": false,
    "changeOrigin": true,
    "logLevel": "debug"
  }
}
```

BASH

```
ng serve --proxy-config proxy.conf.json
```

5. BACKEND — ASP.NET Web API + SQL Server (1 elemento)

5.1. Tecnologias e pacotes NuGet

Pacote	Versão	Papel
<code>Microsoft.EntityFrameworkCore.SqlServer</code>	8.x	ORM + provider SQL Server
<code>Microsoft.EntityFrameworkCore.Tools</code>	8.x	Migrations (<code>Add-Migration</code> , <code>Update-Database</code>)
<code>Microsoft.AspNetCore.Authentication.JwtBearer</code>	8.x	Autenticação JWT
<code>System.IdentityModel.Tokens.Jwt</code>	latest	Geração de tokens JWT
<code>BCrypt.Net-Next</code>	latest	Hash seguro de passwords
<code>AutoMapper.Extensions.Microsoft.DependencyInjection</code>	latest	Mapeamento Entidade ↔ DTO
<code>FluentValidation.AspNetCore</code>	latest	Validação avançada de DTOs
<code>Swashbuckle.AspNetCore</code>	latest	Swagger / OpenAPI

5.2. Estrutura da solução (camadas)

```
NzolaNet.sln
|
|— NzolaNet.Api/                                     ← Camada de Apresentação
|   |   # Endpoints REST
|   |— Controllers/
|   |   |— AuthController.cs
|   |   |— UsersController.cs
|   |   |— PostsController.cs
|   |   |— CommentsController.cs
|   |   |— BazesController.cs
|   |   |— FollowsController.cs
|   |   |— FeedController.cs
|   |   |— NotificationsController.cs
|   |   |— AdminController.cs
|   |— Middleware/
|   |   |— ExceptionMiddleware.cs
|   |   |— RequestLoggingMiddleware.cs
|   |— Extensions/
|   |   |— ServiceCollectionExtensions.cs
|   |— wwwroot/
|   |   |— uploads/
|   |   |   |— photos/
|   |   |   |— media/
|   |— appsettings.json
|   |— appsettings.Development.json
|   |— Program.cs
|
|— NzolaNet.Application/                             ← Camada de Aplicação (Lógica de Negócio)
|   |— Services/
|   |   |— Interfaces/
|   |   |   |— IAuthService.cs
|   |   |   |— IUserService.cs
|   |   |   |— IPostService.cs
|   |   |   |— ICommentService.cs
|   |   |   |— IBazeService.cs
|   |   |   |— IFollowService.cs
|   |   |   |— IFeedService.cs
|   |   |   |— INotificationService.cs
|   |   |   |— IStorageService.cs
|   |   |— Implementations/
|   |   |   |— AuthService.cs
|   |   |   |— UserService.cs
|   |   |   |— PostService.cs
|   |   |   |— CommentService.cs
|   |   |   |— BazeService.cs
|   |   |   |— FollowService.cs
|   |   |   |— FeedService.cs
|   |   |   |— NotificationService.cs
|   |— DTOs/
|   |   |— Auth/                                     # RegisterDto, LoginDto, AuthResponseDto, ForgotPasswordDto, Reset
|   |   |— Users/                                   # UserDto, UserProfileDto, UpdateProfileDto
```

```

├── Posts/           # PostDto, CreatePostDto, UpdatePostDto
├── Comments/        # CommentDto, CreateCommentDto, UpdateCommentDto
├── Bazes/           # BazeToggleResultDto
├── Follows/         # FollowDto
├── Notifications/   # NotificationDto
├── Shared/          # PagedResultDto<T>, MessageDto
├── Mappings/
├──   AutoMapperProfile.cs
├── Validators/      # FluentValidation
├──   RegisterDtoValidator.cs
├──   CreatePostDtoValidator.cs
├──   ...
├── Exceptions/
├──   NotFoundException.cs
├──   BadRequestException.cs
├──   UnauthorizedException.cs
├──   ForbiddenException.cs
├──   ConflictException.cs
├── NzolaNet.Domain/ ← Camada de Domínio (Entidades + contratos)
├──   Entities/
├──     User.cs
├──     Post.cs
├──     Comment.cs
├──     Baze.cs
├──     Follow.cs
├──     Notification.cs
├──     PasswordResetToken.cs
├──   Enums/
├──     NotificationType.cs # Baze, Comment, Follow
├──   Interfaces/
├──     Repositories/
├──       IUserRepository.cs
├──       IPostRepository.cs
├──       ICommentRepository.cs
├──       IBazeRepository.cs
├──       IFollowRepository.cs
├──       INotificationRepository.cs
├──       IUnitOfWork.cs (opcional)
├── NzolaNet.Infrastructure/ ← Camada de Acesso a Dados
├──   Data/
├──     ApplicationDbContext.cs
├──     DbInitializer.cs # Seed (admin inicial)
├──     Configurations/ # Fluent API (constraints, indexes)
├──       UserConfiguration.cs
├──       PostConfiguration.cs
├──       ...
├──   Repositories/
├──     UserRepository.cs
├──     PostRepository.cs
├──     CommentRepository.cs
├──     BazeRepository.cs
├──     FollowRepository.cs
├──     NotificationRepository.cs

```

```
|─ Migrations/                                # Gerado pelo EF Core
|─ Services/
|   |─ JwtTokenService.cs
|   |─ PasswordHasher.cs                    # BCrypt
|   |─ LocalFileStorageService.cs
```

5.3. Base de Dados — Script SQL Server (3FN)

SQL

```
-- =====
-- NzolaNetDB - Script de Criação (SQL Server)
-- =====

CREATE DATABASE NzolaNetDB;
GO
USE NzolaNetDB;
GO

-- =====
-- Users
-- =====

CREATE TABLE Users (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    Username          NVARCHAR(50)  NOT NULL UNIQUE,
    Email             NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash      NVARCHAR(255) NOT NULL,
    FullName          NVARCHAR(100) NOT NULL,
    Bio               NVARCHAR(500) NULL,
    ProfilePhotoUrl   NVARCHAR(500) NULL,
    IsPrivate         BIT NOT NULL DEFAULT 0,
    IsAdmin           BIT NOT NULL DEFAULT 0,
    CreatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    UpdatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);
CREATE INDEX IX_Users_Username ON Users(Username);
CREATE INDEX IX_Users_Email   ON Users(Email);

-- =====
-- Posts
-- =====

CREATE TABLE Posts (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    UserId            INT NOT NULL,
    Content           NVARCHAR(5000) NOT NULL,
    ImageUrl          NVARCHAR(500) NULL,
    VideoUrl          NVARCHAR(500) NULL,
    CreatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    UpdatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    IsDeleted         BIT NOT NULL DEFAULT 0,
    CONSTRAINT FK_Posts_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE CASCADE
);
CREATE INDEX IX_Posts_UserId_CreatedAt ON Posts(UserId, CreatedAt DESC);

-- =====
-- Bazes (likes) - UNIQUE garante 1 baze por user por publicação
-- =====

CREATE TABLE Bazes (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    PostId            INT NOT NULL,
```

```

    UserId    INT NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_Bazes_Posts FOREIGN KEY (PostId) REFERENCES Posts(Id) ON DELETE CASCADE,
    CONSTRAINT FK_Bazes_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE NO ACTION
    CONSTRAINT UQ_Bazes_Post_User UNIQUE (PostId, UserId)
);

-- -----
-- Comments
-- -----

CREATE TABLE Comments (
    Id          INT IDENTITY(1,1) PRIMARY KEY,
    PostId      INT NOT NULL,
    UserId      INT NOT NULL,
    Content     NVARCHAR(1000) NOT NULL,
    CreatedAt   DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    UpdatedAt   DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    IsDeleted   BIT NOT NULL DEFAULT 0,
    CONSTRAINT FK_Comments_Posts FOREIGN KEY (PostId) REFERENCES Posts(Id) ON DELETE CASCADE,
    CONSTRAINT FK_Comments_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE NO ACTION
);
CREATE INDEX IX_Comments_PostId_CreatedAt ON Comments(PostId, CreatedAt);

-- -----
-- Follows (auto-relação Users ↔ Users)
-- -----

CREATE TABLE Follows (
    Id          INT IDENTITY(1,1) PRIMARY KEY,
    FollowerId  INT NOT NULL,
    FollowedId  INT NOT NULL,
    CreatedAt   DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_Follows_Follower FOREIGN KEY (FollowerId) REFERENCES Users(Id) ON DELETE CASCADE,
    CONSTRAINT FK_Follows_Followed FOREIGN KEY (FollowedId) REFERENCES Users(Id) ON DELETE CASCADE,
    CONSTRAINT UQ_Follows_Pair    UNIQUE (FollowerId, FollowedId),
    CONSTRAINT CK_Follows_NotSelf CHECK (FollowerId <> FollowedId)
);
CREATE INDEX IX_Follows_FollowerId ON Follows(FollowerId);
CREATE INDEX IX_Follows_FollowedId ON Follows(FollowedId);

-- -----
-- Notifications
-- -----

CREATE TABLE Notifications (
    Id          INT IDENTITY(1,1) PRIMARY KEY,
    RecipientId INT NOT NULL,           -- quem recebe
    SenderId    INT NOT NULL,          -- quem gerou a ação
    Type        NVARCHAR(20) NOT NULL CHECK (Type IN ('Baze', 'Comment', 'Follow')),
    PostId      INT NULL,              -- opcional (NULL para Follow)
    IsRead      BIT NOT NULL DEFAULT 0,
    CreatedAt   DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_Notif_Recipient FOREIGN KEY (RecipientId) REFERENCES Users(Id) ON DELETE CASCADE,
    CONSTRAINT FK_Notif_Sender    FOREIGN KEY (SenderId)    REFERENCES Users(Id) ON DELETE CASCADE,
    CONSTRAINT FK_Notif_Post      FOREIGN KEY (PostId)      REFERENCES Posts(Id) ON DELETE CASCADE
);
CREATE INDEX IX_Notif_Recipient_IsRead ON Notifications(RecipientId, IsRead);

```

```
-- _____
-- PasswordResetTokens
-- _____

CREATE TABLE PasswordResetTokens (
    Id          INT IDENTITY(1,1) PRIMARY KEY,
    UserId      INT NOT NULL,
    TokenHash   NVARCHAR(255) NOT NULL UNIQUE,
    ExpiresAt   DATETIME2 NOT NULL,
    UsedAt      DATETIME2 NULL,
    CreatedAt   DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_PRT_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE CASCADE
);
```

5.3.1. Relacionamentos (ER)

```
Users  (1) ——— (N) Posts
Users  (1) ——— (N) Comments
Users  (1) ——— (N) Bazes
Users  (N) ——— (N) Users      (via Follows)
Posts  (1) ——— (N) Comments
Posts  (1) ——— (N) Bazes
Users  (1) ——— (N) Notifications (RecipientId)
Users  (1) ——— (N) Notifications (SenderId)
Posts  (0..1) — (N) Notifications
Users  (1) ——— (N) PasswordResetTokens
```

5.4. Entidades (EF Core — Code-First)

CSHARP

```
// Domain/Entities/User.cs
public class User {
    public int Id { get; set; }
    public string Username { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string? Bio { get; set; }
    public string? ProfilePhotoUrl { get; set; }
    public bool IsPrivate { get; set; }
    public bool IsAdmin { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

    public ICollection<Post> Posts { get; set; } = [];
    public ICollection<Baze> Bazes { get; set; } = [];
    public ICollection<Comment> Comments { get; set; } = [];
    public ICollection<Follow> Followers { get; set; } = []; // quem me segue
    public ICollection<Follow> Following { get; set; } = []; // quem eu sigo
    public ICollection<Notification> NotificationsReceived { get; set; } = [];
    public ICollection<Notification> NotificationsSent { get; set; } = [];
}

// Domain/Entities/Post.cs
public class Post {
    public int Id { get; set; }
    public int UserId { get; set; }
    public User User { get; set; } = null!;
    public string Content { get; set; } = string.Empty;
    public string? ImageUrl { get; set; }
    public string? VideoUrl { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    public bool IsDeleted { get; set; }

    public ICollection<Baze> Bazes { get; set; } = [];
    public ICollection<Comment> Comments { get; set; } = [];
}

// Domain/Entities/Baze.cs
public class Baze {
    public int Id { get; set; }
    public int PostId { get; set; }
    public Post Post { get; set; } = null!;
    public int UserId { get; set; }
    public User User { get; set; } = null!;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

// Domain/Entities/Comment.cs
```

```

public class Comment {
    public int Id { get; set; }
    public int PostId { get; set; }
    public Post Post { get; set; } = null!;
    public int UserId { get; set; }
    public User User { get; set; } = null!;
    public string Content { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    public bool IsDeleted { get; set; }
}

// Domain/Entities/Follow.cs
public class Follow {
    public int Id { get; set; }
    public int FollowerId { get; set; }
    public User Follower { get; set; } = null!;
    public int FollowedId { get; set; }
    public User Followed { get; set; } = null!;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

// Domain/Entities/Notification.cs
public class Notification {
    public int Id { get; set; }
    public int RecipientId { get; set; }
    public User Recipient { get; set; } = null!;
    public int SenderId { get; set; }
    public User Sender { get; set; } = null!;
    public NotificationType Type { get; set; }
    public int? PostId { get; set; }
    public Post? Post { get; set; }
    public bool IsRead { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

public enum NotificationType { Baze, Comment, Follow }

```

5.5. ApplicationDbContext

CSHARP

```
// Infrastructure/Data/ApplicationDbContext.cs
public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : DbContext
{
    public DbSet<User> Users => Set<User>();
    public DbSet<Post> Posts => Set<Post>();
    public DbSet<Baze> Bazes => Set<Baze>();
    public DbSet<Comment> Comments => Set<Comment>();
    public DbSet<Follow> Follows => Set<Follow>();
    public DbSet<Notification> Notifications => Set<Notification>();
    public DbSet<PasswordResetToken> PasswordResetTokens => Set<PasswordResetToken>();

    protected override void OnModelCreating(ModelBuilder mb)
    {
        // Unici dades
        mb.Entity<User>().HasIndex(u => u.Username).IsUnique();
        mb.Entity<User>().HasIndex(u => u.Email).IsUnique();
        mb.Entity<Baze>().HasIndex(b => new { b.PostId, b.UserId }).IsUnique();
        mb.Entity<Follow>().HasIndex(f => new { f.FollowerId, f.FollowedId }).IsUnique();

        // Auto-refer ncia Follows (evitar m ltiplas cascatas)
        mb.Entity<Follow>()
            .HasOne(f => f.Follower).WithMany(u => u.Following)
            .HasForeignKey(f => f.FollowerId).OnDelete(DeleteBehavior.NoAction);

        mb.Entity<Follow>()
            .HasOne(f => f.Followed).WithMany(u => u.Followers)
            .HasForeignKey(f => f.FollowedId).OnDelete(DeleteBehavior.NoAction);

        mb.Entity<Follow>()
            .ToTable(t => t.HasCheckConstraint("CK_Follows_NotSelf", "[FollowerId] <> [FollowedId]"));

        // Notifications (evitar m ltiplas cascatas)
        mb.Entity<Notification>()
            .HasOne(n => n.Recipient).WithMany(u => u.NotificationsReceived)
            .HasForeignKey(n => n.RecipientId).OnDelete(DeleteBehavior.NoAction);

        mb.Entity<Notification>()
            .HasOne(n => n.Sender).WithMany(u => u.NotificationsSent)
            .HasForeignKey(n => n.SenderId).OnDelete(DeleteBehavior.NoAction);

        // Convers o de enum para string (legibilidade na BD)
        mb.Entity<Notification>()
            .Property(n => n.Type).HasConversion<string>().HasMaxLength(20);
    }
}
```

5.6. DTOs essenciais

CSHARP

```
// DTOs/Auth
public record RegisterDto(string Username, string Email, string Password, string FullName);
public record LoginDto(string Email, string Password);
public record AuthResponseDto(string Token, DateTime Expiration, UserDto User);
public record ForgotPasswordDto(string Email);
public record ResetPasswordDto(string Token, string NewPassword);

// DTOs/Users
public record UserDto(
    int Id, string Username, string Email, string FullName,
    string? Bio, string? ProfilePhotoUrl, bool IsPrivate, bool IsAdmin);

public record UserProfileDto(
    int Id, string Username, string FullName, string? Bio,
    string? ProfilePhotoUrl, bool IsPrivate,
    int FollowersCount, int FollowingCount, int PostsCount,
    bool IsFollowing);

public record UpdateProfileDto(string FullName, string? Bio, bool IsPrivate);

// DTOs/Posts
public record PostDto(
    int Id, int AuthorId, string AuthorName, string AuthorUsername,
    string? AuthorPhotoUrl, string Content,
    string? ImageUrl, string? VideoUrl,
    int BazesCount, int CommentsCount, bool UserHasBazed,
    DateTime CreatedAt, DateTime UpdatedAt);

public record CreatePostDto(string Content, IFormFile? Image, IFormFile? Video);
public record UpdatePostDto(string Content, IFormFile? Image, IFormFile? Video);

// DTOs/Comments
public record CommentDto(
    int Id, int PostId, int AuthorId, string AuthorName,
    string? AuthorPhotoUrl, string Content,
    bool IsEditable, DateTime CreatedAt);

public record CreateCommentDto(string Content);
public record UpdateCommentDto(string Content);

// DTOs/Bazes
public record BazeToggleResultDto(int BazesCount, bool UserHasBazed);

// DTOs/Notifications
public record NotificationDto(
    int Id, string Type, int FromUserId, string FromUsername,
    string? FromPhotoUrl, int? PostId, bool IsRead, DateTime CreatedAt);

// DTOs/Shared
public record PagedResultDto<T>(
```

```
IEnumerable<T> Items, int Page, int PageSize, int TotalCount, int TotalPages);

public record MessageDto(string Message);
```

5.7. Endpoints REST (Controllers)

Base URL: `http://localhost:5000/api`

AuthController — `/api/auth`

Método	Rota	Auth	Body	Resposta
POST	<code>/register</code>	✗	<code>RegisterDto</code>	<code>AuthResponseDto</code>
POST	<code>/login</code>	✗	<code>LoginDto</code>	<code>AuthResponseDto</code>
POST	<code>/forgot-password</code>	✗	<code>ForgotPasswordDto</code>	<code>MessageDto</code>
POST	<code>/reset-password</code>	✗	<code>ResetPasswordDto</code>	<code>MessageDto</code>

UsersController — `/api/users`

Método	Rota	Auth	Resposta
GET	<code>/id</code>	✓	<code>UserProfileDto</code>
PUT	<code>/me</code>	✓	<code>UserProfileDto</code>
PUT	<code>/me/photo</code>	✓ (multipart)	<code>UserProfileDto</code>
POST	<code>/id/follow</code>	✓	<code>MessageDto</code>
DELETE	<code>/id/follow</code>	✓	<code>MessageDto</code>
GET	<code>/id/followers</code>	✓	<code>PagedResultDto<UserDto></code>
GET	<code>/id/following</code>	✓	<code>PagedResultDto<UserDto></code>

PostsController — /api/posts

Método	Rota	Auth	Resposta
POST	/	✓ (multipart)	PostDto
GET	/ {id}	✓	PostDto
PUT	/ {id}	✓ (dono)	PostDto
DELETE	/ {id}	✓ (dono)	MessageDto
GET	/user/{userId}	✓	PagedResultDto<PostDto>

BazesController — /api/posts/{id}/baze

Método	Rota	Auth	Resposta
POST	/posts/{id}/baze	✓ (idempotente, toggle)	BazeToggleResultDto

CommentsController

Método	Rota	Auth	Resposta
GET	/posts/{postId}/comments	✓	PagedResultDto<CommentDto>
POST	/posts/{postId}/comments	✓	CommentDto
PUT	/comments/{id}	✓ (dono)	CommentDto
DELETE	/comments/{id}	✓ (dono ou admin)	MessageDto

FeedController — /api/feed

Método	Rota	Auth	Resposta
GET	/?page=1&pageSize=10	✓	PagedResultDto<PostDto>

NotificationsController — /api/notifications

Método	Rota	Auth	Resposta
GET	/?page=1&pageSize=20	✓	PagedResultDto<NotificationDto>
GET	/unread-count	✓	int
PUT	/id/read	✓	MessageDto
PUT	/read-all	✓	MessageDto

AdminController — /api/admin

Método	Rota	Auth	Resposta
DELETE	/comments/{id}	✓ (admin)	MessageDto

Códigos HTTP padronizados

- **200 OK** — sucesso em GET/PUT/DELETE
- **201 Created** — sucesso em POST que cria recurso
- **202 Accepted** — operação aceite (ex.: recuperação de senha enviada)
- **400 Bad Request** — validação de DTO falhou
- **401 Unauthorized** — sem token ou token inválido
- **403 Forbidden** — autenticado mas sem permissão
- **404 Not Found** — recurso inexistente
- **409 Conflict** — conflito (ex.: já segue, username em uso)
- **500 Internal Server Error** — erro inesperado

5.8. Lógica de negócio crítica (Services)

5.8.1. Toggle de Baze (com prevenção de duplicados)

CSHARP

```
public async Task<BazeToggleResultDto> ToggleAsync(int postId, int userId)
{
    var existing = await _bazeRepo.GetAsync(userId, postId);

    if (existing is not null) {
        await _bazeRepo.RemoveAsync(existing);
    } else {
        await _bazeRepo.AddAsync(new Baze { UserId = userId, PostId = postId });
    }

    var post = await _postRepo.GetByIdAsync(postId)
        ?? throw new NotFoundException("Publicação não encontrada.");

    // Não notificar o próprio autor
    if (post.UserId != userId)
        await _notificationService.CreateAsync(
            recipientId: post.UserId,
            senderId: userId,
            type: NotificationType.Baze,
            postId: postId);
}

var count = await _bazeRepo.CountByPostAsync(postId);
return new BazeToggleResultDto(count, existing is null);
}
```

5.8.2. Validação de autoria em edição/exclusão

CSHARP

```
public async Task<PostDto> UpdateAsync(int postId, UpdatePostDto dto, int currentUserId)
{
    var post = await _postRepo.GetByIdAsync(postId)
        ?? throw new NotFoundException("Publicação não encontrada.");

    if (post.UserId != currentUserId)
        throw new ForbiddenException("Apenas o autor pode editar esta publicação.");

    post.Content = dto.Content;
    post.UpdatedAt = DateTime.UtcNow;
    // tratar upload de Image/Video se fornecidos

    await _postRepo.UpdateAsync(post);
    return _mapper.Map<PostDto>(post);
}
```

5.8.3. Feed com paginação (publicações de seguidos + próprias)

```
CSHARP

public async Task<PagedResultDto<PostDto>> GetFeedAsync(int userId, int page, int pageSize)
{
    var followedIds = await _followRepo.GetFollowedIdsAsync(userId);

    var query = _ctx.Posts
        .Include(p => p.User)
        .Where(p => !p.IsDeleted &&
            (p.UserId == userId || followedIds.Contains(p.UserId)))
        .OrderByDescending(p => p.CreatedAt);

    var total = await query.CountAsync();
    var items = await query
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .Select(p => new PostDto(
            p.Id, p.UserId, p.User.FullName, p.User.Username,
            p.User.ProfilePhotoUrl, p.Content, p.ImageUrl, p.VideoUrl,
            p.Bazes.Count, p.Comments.Count,
            p.Bazes.Any(b => b.UserId == userId),
            p.CreatedAt, p.UpdatedAt))
        .ToListAsync();

    return new PagedResultDto<PostDto>(items, page, pageSize, total,
        (int)Math.Ceiling(total / (double)pageSize));
}
```

5.8.4. Perfil privado

```
CSHARP

public async Task<UserProfileDto> GetProfileAsync(int targetId, int requesterId)
{
    var target = await _userRepo.GetByIdAsync(targetId)
        ?? throw new NotFoundException("Utilizador não encontrado.");

    var isFollowing = await _followRepo.ExistsAsync(requesterId, targetId);

    if (target.IsPrivate && target.Id != requesterId && !isFollowing) {
        // perfil minimal (sem posts/seguidores)
        return new UserProfileDto(target.Id, target.Username, target.FullName,
            null, target.ProfilePhotoUrl, true, 0, 0, 0, false);
    }

    return _mapper.Map<UserProfileDto>(target) with { IsFollowing = isFollowing };
}
```

5.8.5. Moderação por administrador

CSHARP

```
public async Task DeleteByAdminAsync(int commentId, int adminUserId)
{
    var admin = await _userRepo.GetByIdAsync(adminUserId);
    if (admin is null || !admin.IsAdmin)
        throw new ForbiddenException("Apenas administradores podem moderar comentários.");

    var comment = await _commentRepo.GetByIdAsync(commentId)
        ?? throw new NotFoundException("Comentário não encontrado.");

    comment.IsDeleted = true;
    await _commentRepo.UpdateAsync(comment);
}
```

5.9. Upload de ficheiros

CSHARP

```
public class LocalFileStorageService : IStorageService
{
    private static readonly string[] _imgExt = [".jpg", ".jpeg", ".png", ".webp"];
    private static readonly string[] _vidExt = [".mp4", ".webm", ".mov"];
    private const long MaxImageSize = 10 * 1024 * 1024; // 10 MB
    private const long MaxVideoSize = 50 * 1024 * 1024; // 50 MB

    public async Task<string> SaveAsync(IFormFile file, string folder)
    {
        var ext = Path.GetExtension(file.FileName).ToLowerInvariant();
        var isImage = _imgExt.Contains(ext);
        var isVideo = _vidExt.Contains(ext);

        if (!isImage && !isVideo)
            throw new BadRequestException("Formato de ficheiro não suportado.");
        if (isImage && file.Length > MaxImageSize)
            throw new BadRequestException("Imagem demasiado grande (máx. 10 MB).");
        if (isVideo && file.Length > MaxVideoSize)
            throw new BadRequestException("Vídeo demasiado grande (máx. 50 MB).");

        var fileName = $"{Guid.NewGuid()}{ext}";
        var dir = Path.Combine("wwwroot", "uploads", folder);
        Directory.CreateDirectory(dir);
        var fullPath = Path.Combine(dir, fileName);

        await using var stream = new FileStream(fullPath, FileMode.Create);
        await file.CopyToAsync(stream);

        return $"/uploads/{folder}/{fileName}";
    }
}
```

5.10. Program.cs (configuração completa)

CSHARP

```
var builder = WebApplication.CreateBuilder(args);

// EF Core + SQL Server
builder.Services.AddDbContext<ApplicationDbContext>(opt =>
    opt.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// JWT
var jwtKey = builder.Configuration["Jwt:Key"]!;
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opt => {
        opt.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero
        };
    });

builder.Services.AddAuthorization(opt => {
    opt.AddPolicy("AdminOnly", p => p.RequireClaim("IsAdmin", "True"));
});

// CORS
builder.Services.AddCors(opt => opt.AddPolicy("Angular", p => p
    .WithOrigins("http://localhost:4200")
    .AllowAnyHeader().AllowAnyMethod().AllowCredentials()));

// AutoMapper + FluentValidation
builder.Services.AddAutoMapper(typeof(AutoMapperProfile).Assembly);
builder.Services.AddValidatorsFromAssembly(typeof(RegisterDtoValidator).Assembly);
builder.Services.AddFluentValidationAutoValidation();

// DI - Repositórios
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IPostRepository, PostRepository>();
builder.Services.AddScoped<IBazeRepository, BazeRepository>();
builder.Services.AddScoped<ICommentRepository, CommentRepository>();
builder.Services.AddScoped<IFollowRepository, FollowRepository>();
builder.Services.AddScoped<INotificationRepository, NotificationRepository>();

// DI - Serviços
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IPostService, PostService>();
builder.Services.AddScoped<ICommentService, CommentService>();
builder.Services.AddScoped<IBazeService, BazeService>();
```

```

builder.Services.AddScoped<IFollowService, FollowService>();
builder.Services.AddScoped<IFeedService, FeedService>();
builder.Services.AddScoped<INotificationService, NotificationService>();
builder.Services.AddScoped<IStorageService, LocalFileStorageService>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c => {
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "NzolaNet API", Version = "v1" });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme { /* JWT */ });
});

var app = builder.Build();

app.UseMiddleware<ExceptionMiddleware>();
app.UseSwagger();
app.UseSwaggerUI();
app.UseCors("Angular");
app.UseAuthentication();
app.UseAuthorization();
app.UseStaticFiles(); // servir wwwroot/uploads
app.MapControllers();

// Seed (admin inicial)
using (var scope = app.Services.CreateScope())
{
    var ctx = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    ctx.Database.Migrate();
    await DbInitializer.SeedAsync(ctx);
}

app.Run();

```

5.11. appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=NzolaNetDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Key": "NzolaNet_SuperSecretKey_PelosMenos32Caracteres!!",
    "Issuer": "NzolaNetApi",
    "Audience": "NzolaNetClient",
    "ExpirationMinutes": 120
  },
  "AllowedHosts": "*"
}

```

5.12. Comandos úteis (backend)

POWERSHELL

```
# Criar a solução e projetos
dotnet new sln -n NzolaNet
dotnet new webapi -n NzolaNet.Api
dotnet new classlib -n NzolaNet.Application
dotnet new classlib -n NzolaNet.Domain
dotnet new classlib -n NzolaNet.Infrastructure
dotnet sln add NzolaNet.Api NzolaNet.Application NzolaNet.Domain NzolaNet.Infrastructure

# Referências entre projetos
dotnet add NzolaNet.Api/NzolaNet.Api.csproj          reference NzolaNet.Application/NzolaNet.Application
dotnet add NzolaNet.Api/NzolaNet.Api.csproj          reference NzolaNet.Infrastructure/NzolaNet.Infrastructure
dotnet add NzolaNet.Application/NzolaNet.Application.csproj reference NzolaNet.Domain/NzolaNet.Domain
dotnet add NzolaNet.Infrastructure/NzolaNet.Infrastructure.csproj reference NzolaNet.Domain/NzolaNet.Domain

# Pacotes NuGet (executar dentro do projeto correto)
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package BCrypt.Net-Next
dotnet add package AutoMapper.Extensions.Microsoft.DependencyInjection
dotnet add package FluentValidation.AspNetCore
dotnet add package Swashbuckle.AspNetCore

# Migrations
dotnet ef migrations add InitialCreate --project NzolaNet.Infrastructure --startup-project NzolaNet.Api
dotnet ef database update --project NzolaNet.Infrastructure --startup-project NzolaNet.Api

# Executar
dotnet run --project NzolaNet.Api
```

6. Contrato da API (Frontend ↔ Backend)

Este contrato deve viver em `docs/API_CONTRACT.md` no repositório e ser **atualizado em conjunto** sempre que algum endpoint mude. Frontend e backend consultam este ficheiro como fonte da verdade.

API Contract – NzolaNet (v1)

Base URL (dev): `http://localhost:5000/api`

Autenticação: `Authorization: Bearer <JWT>`

Auth

POST	<code>/auth/register</code>	body: RegisterDto	→ 201 AuthResponseDto
POST	<code>/auth/login</code>	body: LoginDto	→ 200 AuthResponseDto
POST	<code>/auth/forgot-password</code>	body: ForgotPasswordDto	→ 202 MessageDto
POST	<code>/auth/reset-password</code>	body: ResetPasswordDto	→ 200 MessageDto

Users

GET	<code>/users/{id}</code>		→ 200 UserProfileDto
PUT	<code>/users/me</code>	body: UpdateProfileDto	→ 200 UserProfileDto
PUT	<code>/users/me/photo</code>	multipart: photo	→ 200 UserProfileDto
POST	<code>/users/{id}/follow</code>		→ 200 MessageDto
DELETE	<code>/users/{id}/follow</code>		→ 200 MessageDto
GET	<code>/users/{id}/followers?page=&pageSize=</code>		→ 200 PagedResultDto<UserDto>
GET	<code>/users/{id}/following?page=&pageSize=</code>		→ 200 PagedResultDto<UserDto>

Posts

POST	<code>/posts</code>	multipart: content, image?, video?	→ 201 PostDto
GET	<code>/posts/{id}</code>		→ 200 PostDto
PUT	<code>/posts/{id}</code>	multipart: content, image?, video?	→ 200 PostDto
DELETE	<code>/posts/{id}</code>		→ 200 MessageDto
GET	<code>/posts/user/{userId}?page=&pageSize=</code>		→ 200 PagedResultDto<PostDto>

Bazes

POST	<code>/posts/{id}/baze</code>		→ 200 BazeToggleResultDto (toggle)
------	-------------------------------	--	------------------------------------

Comments

GET	<code>/posts/{postId}/comments?page=&pageSize=</code>		→ 200 PagedResultDto<CommentDto>
POST	<code>/posts/{postId}/comments</code>	body: CreateCommentDto	→ 201 CommentDto
PUT	<code>/comments/{id}</code>	body: UpdateCommentDto	→ 200 CommentDto
DELETE	<code>/comments/{id}</code>		→ 200 MessageDto

Feed

GET	<code>/feed?page=&pageSize=</code>		→ 200 PagedResultDto<PostDto>
-----	--	--	-------------------------------

Notifications

GET	<code>/notifications?page=&pageSize=</code>		→ 200 PagedResultDto<NotificationDto>
GET	<code>/notifications/unread-count</code>		→ 200 int
PUT	<code>/notifications/{id}/read</code>		→ 200 MessageDto
PUT	<code>/notifications/read-all</code>		→ 200 MessageDto

Admin

DELETE	<code>/admin/comments/{id}</code>		→ 200 MessageDto (role: Admin)
--------	-----------------------------------	--	--------------------------------

7. Git e GitHub — Estratégia de Colaboração

7.1. Setup inicial (uma única vez, feito por um líder do grupo)

Passo 1 — Criar o repositório no GitHub

1. Aceder a <https://github.com/new>.
2. Nome do repositório: `nzolanet`.
3. Visibilidade: **Private** (recomendado para projetos académicos).
4. Marcar **Add a README file** e **Add .gitignore** (escolher o template **Node** e depois acrescentar manualmente regras para .NET).
5. Clicar em **Create repository**.

Passo 2 — Convidar os membros do grupo

1. No repositório, ir a **Settings** → **Collaborators and teams** → **Add people**.
2. Adicionar pelo username do GitHub dos 3 colegas (e o teu, se não fores o criador).
3. Atribuir-lhes role **Write**.
4. Cada colaborador recebe um e-mail e precisa de **aceitar o convite** antes de poder fazer push.

Passo 3 — Clonar e preparar a estrutura local

BASH

```
git clone https://github.com/<organizacao-ou-utilizador>/nzolanet.git
cd nzolanet

# Estrutura de pastas
mkdir backend frontend docs
echo "# NzolaNet" > README.md

# .gitignore (fundir Node + .NET + IDEs)
cat > .gitignore << 'EOF'
# Node / Angular
node_modules/
dist/
.angular/
*.log
.env
.env.local

# .NET
bin/
obj/
*.user
*.suo
appsettings.Development.json
appsettings.Production.json
wwwroot/uploads/

# IDE
.vs/
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db
EOF

git add .
git commit -m "chore: estrutura inicial do monorepo (backend, frontend, docs)"
git push origin main
```

Passo 4 — Criar a branch `develop`

BASH

```
git checkout -b develop
git push -u origin develop
```

Passo 5 — Proteger as branches no GitHub

1. **Settings** → **Branches** → **Add branch protection rule**.
2. Branch name pattern: `main`.
 - ☒ *Require a pull request before merging*
 - ☒ *Require approvals* (1 aprovação mínima)
 - ☒ *Do not allow bypassing the above settings*
3. Repetir para `develop` com regras semelhantes (mas pode-se permitir merge sem aprovação noutros momentos para agilidade — combinem em grupo).
4. Em **Settings** → **General** → **Default branch**, mudar o default para `develop` (assim novos clones começam nela).

7.2. Estratégia de branches (Git Flow simplificado)

```
main                ← versão estável (entregas). Protegida. Nunca commit direto.
|
develop             ← integração contínua. Todas as PRs vão para aqui.
|
├─ backend/feature/auth                ← Willfredy
├─ backend/feature/posts
├─ backend/feature/comments
├─ backend/feature/feed
├─ backend/feature/notifications
|
├─ frontend/feature/auth                ← Emer (FE-Dev 1)
├─ frontend/feature/feed                ← Jeovani (FE-Dev 2)
└─ frontend/feature/comments-notifications ← Manuel (FE-Dev 3)
```

Convenção de nomes de branch

Tipo	Formato	Exemplo
Feature backend	<code>backend/feature/<nome-curto></code>	<code>backend/feature/posts-crud</code>
Feature frontend	<code>frontend/feature/<nome-curto></code>	<code>frontend/feature/feed-infinite-scroll</code>
Bug fix	<code>fix/<escopo>-<descricao></code>	<code>fix/baze-toggle-count</code>
Hotfix	<code>hotfix/<urgente></code>	<code>hotfix/jwt-expiry</code>
Documentação	<code>docs/<assunto></code>	<code>docs/api-contract-update</code>

7.3. Workflow diário — passo a passo

Início do dia (TODOS os membros)

BASH

```
# 1) Posicionar-se na develop e ir buscar as últimas alterações
git checkout develop
git pull origin develop
```

Começar uma nova tarefa

BASH

```
# 2) Criar branch própria a partir de develop
git checkout -b frontend/feature/auth      # exemplo: Emer (Auth)
# ou
git checkout -b backend/feature/posts-crud # exemplo Willfredy
```

Trabalhar e commitar (frequência alta, mensagens claras)

Convenção de commits (Conventional Commits):

```
feat(<escopo>):      nova funcionalidade
fix(<escopo>):        correção de bug
refactor(<escopo>):  melhoria sem mudança de comportamento
docs(<escopo>):       alteração de documentação
chore(<escopo>):     config, dependências, manutenção
test(<escopo>):       testes
style(<escopo>):      formatação, espaços, etc. (sem lógica)
```

Exemplos:

BASH

```
git add .
git commit -m "feat(backend/auth): implementa login e registo com JWT"
git commit -m "feat(frontend/feed): adiciona infinite scroll com paginação"
git commit -m "fix(backend/bazes): garante unicidade no toggle"
git commit -m "docs(api): atualiza contrato de notificações"
```

Enviar para o GitHub (primeira vez na branch)

BASH

```
git push -u origin frontend/feature/auth
```

Enviar atualizações na mesma branch

```
git push
```

BASH

Antes de abrir o PR — sincronizar com `develop`

```
# Garantir que a tua branch está atualizada com a develop antes do PR
git checkout develop
git pull origin develop
git checkout frontend/feature/auth
git merge develop          # resolver conflitos localmente se existirem
# (alternativa mais limpa: git rebase develop)
git push
```

BASH

7.4. Pull Requests (PRs)

1. No GitHub, abrir um **Pull Request** da tua branch para `develop`.
2. **Título:** segue a convenção do commit principal.
 - `feat(backend/auth): Implementa autenticação JWT`
3. **Descrição (template recomendado):**

```
## O que foi feito?
- Endpoint POST /auth/register
- Endpoint POST /auth/login com JWT
- Validações com FluentValidation

## Como testar?
1. Iniciar a API (`dotnet run --project NzolaNet.Api`)
2. Abrir Swagger em https://localhost:5001/swagger
3. POST /auth/register com payload de exemplo
4. POST /auth/login com as mesmas credenciais → deve retornar token

## Contrato API afetado?
- [x] Sim - ver docs/API_CONTRACT.md secção Auth
- [ ] Não

## Checklist
- [x] Build passa sem erros
- [x] Endpoints documentados no Swagger
- [x] Sem credenciais commitadas
```

MARKDOWN

4. **Atribuir um revisor** (outro membro do grupo). **Mínimo 1 aprovação** antes do merge.

5. **Estratégia de merge:** `Squash and merge` para manter o histórico de `develop` limpo (1 commit por feature).
6. Após merge, **apagar a branch remota** (botão no GitHub) para reduzir ruído.

7.5. Promoção `develop` → `main` (entregas)

Apenas em marcos importantes (2.^a Parcelar, Exame Final):

```
git checkout main
git pull origin main
git merge develop --no-ff -m "release(v1.0): 2ª Parcelar - Users, Posts, Comments"
git push origin main

# Criar tag de versão
git tag v1.0-parcelar
git push origin --tags
```

BASH

7.6. Prevenção de conflitos

Risco	Prevenção
Dois devs editam o mesmo ficheiro	Divisão clara de módulos/pastas; comunicação no grupo
Conflitos em <code>app.module.ts</code> / <code>app-routing.module.ts</code>	Definir rotas e módulos no início; só um dev altera por vez (PR imediato)
Conflitos em <code>package.json</code> / <code>.csproj</code>	"Dono" único por ficheiro; resolução manual com revisão
Backend muda DTO que frontend usa	1. Atualizar <code>API_CONTRACT.md</code> no PR. 2. Avisar grupo. 3. Frontend só faz merge depois de atualizar modelos TypeScript
Branch desatualizada gera muitos conflitos	<code>git pull origin develop</code> diário + merge/rebase antes de abrir PR

7.7. Comandos Git essenciais (referência rápida)

BASH

```
# Estado atual e histórico
git status
git log --oneline --graph --decorate --all

# Branches
git branch                # listar locais
git branch -r             # listar remotas
git branch -d <nome>      # apagar local
git push origin --delete <nome> # apagar remota

# Sincronização
git fetch --all           # buscar tudo sem merge
git pull --rebase origin develop # rebase em vez de merge

# Desfazer
git restore <ficheiro>    # descartar alterações não commitadas
git restore --staged <ficheiro> # tirar do staging
git reset --soft HEAD~1   # desfazer último commit (mantém alterações)
git reset --hard HEAD~1   # desfazer último commit (PERDE alterações)
git revert <hash>         # cria commit que reverte outro (seguro)

# Stash (guardar trabalho temporariamente)
git stash                # guardar
git stash pop            # restaurar e remover do stash
git stash list           # listar stashes

# Resolver conflitos
# 1) editar manualmente os ficheiros com <====, >>>>>>
# 2) git add <ficheiros-resolvidos>
# 3) git commit (ou git rebase --continue)
```

7.8. Ferramentas auxiliares

- **Swagger UI** (<https://localhost:5001/swagger>) — documentação viva da API, útil para o frontend testar.
- **Postman** — exportar collection a partir do Swagger e versionar em [docs/postman/](#).
- **dbdiagram.io** — manter o ERD em [docs/database/erd.png](#) atualizado.
- **GitHub Projects** (Kanban) — recomendado: criar colunas *Backlog* → *Em curso* → *Em review* → *Concluído* e mover issues.
- **GitHub Issues** — uma issue por funcionalidade; referenciar nos commits/PRs com [#numero](#).

8. Cronograma de Entregas

Datas oficiais (enunciado): 2.ª Parcelar (Users, Posts, Comments) → Exame de Época Normal (aplicação completa + relatório).

Fase 1 — 2.ª Parcelar (Semanas 1–4)

Semana	Backend (Willfredy)	Frontend (3 devs)
1	Setup solução + EF Core + DB schema + Migration inicial + AuthController (register/login + JWT)	Setup Angular + módulo <code>auth</code> (login/register) + guards + interceptor JWT + layouts
2	UserController (CRUD perfil, follow/unfollow) + upload de foto	<code>edit-profile/</code> + <code>profile-page/</code> + integração com auth + componentes <code>shared/</code> (navbar, avatar)
3	PostsController (CRUD + upload media) + CommentsController (CRUD)	<code>feed-page/</code> + <code>create-post/</code> + <code>post-card</code> + <code>comment-list/</code> + <code>comment-form/</code>
4	Testes, correções, polimento, relatório parcial, Swagger documentado	Responsividade, validações, integração end-to-end, testes em mobile

Fase 2 — Exame de Época Normal (Semanas 5–8)

Semana	Backend	Frontend
5	BazesController (toggle) + FeedController (paginado)	Toggle de base visual + feed cronológico + paginação/infinite scroll
6	FollowsController + Notifications (geração automática)	<code>notification-bell</code> + <code>notifications-page</code> + polling de contagem
7	Perfil público/privado + AdminController (moderação)	UI condicional para perfil privado + painel admin
8	Hardening de segurança, performance, relatório final	UI polish, acessibilidade, testes de regressão, relatório final

9. Checklist Final para 20/20

Backend ✓

- ☐ Solução .NET com 4 projetos (Api, Application, Domain, Infrastructure)
- ☐ Arquitetura em camadas: **Controller** → **Service** → **Repository**
- ☐ **DTOs em todos os endpoints** (nunca expor entidades diretamente)
- ☐ Entity Framework Core Code-First + Migrations + Seed (admin inicial)
- ☐ Base de dados SQL Server **normalizada (3FN)** com índices nos campos pesquisados
- ☐ Constraint **UNIQUE** em `Bazes(PostId, UserId)` e `Follows(FollowerId, FollowedId)`
- ☐ Constraint **CHECK** em `Follows` (não seguir a si próprio)
- ☐ **JWT Authentication** funcional (register + login + refresh opcional)
- ☐ Hash de password com **BCrypt**
- ☐ Recuperação de senha com **token + expiração**
- ☐ Atributo `[Authorize]` em todos os endpoints privados;
`[Authorize(Policy="AdminOnly")]` em moderação
- ☐ **CRUD completo** de Users, Posts, Comments
- ☐ **Toggle de Baze** com unicidade garantida (constraint + lógica)
- ☐ **Feed paginado** (publicações próprias + de seguidos, ordem cronológica)
- ☐ **Follow/Unfollow** entre utilizadores
- ☐ **Notificações automáticas** (baze, comentário, novo seguidor) — nunca auto-notificar
- ☐ **Moderação** de comentários por admin
- ☐ **Privacidade** de perfil (público/privado com regra de seguidores)
- ☐ **Upload de imagens e vídeos** com validação de tipo e tamanho
- ☐ **CORS** configurado para `http://localhost:4200`
- ☐ **Swagger** completo (com Bearer auth)
- ☐ **Middleware global de exceções** (respostas JSON padronizadas)
- ☐ **AutoMapper** para mapeamento Entidade ↔ DTO
- ☐ **FluentValidation** nos DTOs de entrada
- ☐ Códigos HTTP corretos (200, 201, 400, 401, 403, 404, 409)

Frontend ✓

- ☐ Estrutura modular (`core/`, `shared/`, `features/`, `layouts/`)
- ☐ **Módulos lazy-loaded** com `loadChildren`
- ☐ **AuthGuard** + **GuestGuard** + (opcional) **AdminGuard**
- ☐ **JwtInterceptor** + **ErrorInterceptor**
- ☐ **Modelos TypeScript** que espelham os DTOs do backend
- ☐ **Reactive Forms** com validação client-side em todos os formulários

- ☐ **Feed cronológico** com paginação ou infinite scroll
- ☐ **Upload de imagem/vídeo** com pré-visualização e validação
- ☐ **Toggle de base** com **atualização otimista da UI**
- ☐ **Componentes reutilizáveis** (`post-card` , `user-avatar` , `notification-bell` , `comment-item`)
- ☐ **Perfil privado** com mensagem apropriada para não seguidores
- ☐ **Notificações** com contador e marcação como lida
- ☐ **Botões Editar/Apagar** condicionais (só para o autor)
- ☐ **Interface responsiva** (mobile-first, breakpoints testados)
- ☐ **Feedback visual** (spinners, toasts, skeleton loaders)
- ☐ **Acessibilidade** (`alt` , `aria-label` , navegação por teclado)
- ☐ **Pipes** (`timeAgo` , `safeUrl`) para apresentação
- ☐ **OnPush** change detection nos componentes presentacionais

Git e Documentação ✓

- ☐ Repositório monorepo (`backend/` , `frontend/` , `docs/`)
- ☐ Branches protegidas (`main` e `develop`)
- ☐ **Branch por feature** por developer
- ☐ **PRs com revisão obrigatória** antes de merge
- ☐ **Convenção de commits** (Conventional Commits) respeitada
- ☐ **Tags de versão** (`v1.0-parcelar` , `v2.0-final`)
- ☐ `docs/API_CONTRACT.md` mantido **atualizado**
- ☐ `docs/database/erd.png` com diagrama ER
- ☐ `README.md` com instruções de setup (clone, install, run)
- ☐ **Relatório técnico** final em `docs/relatorio-final.pdf` com:
 - Análise de requisitos
 - Arquitetura adotada e justificação
 - Modelo de dados (ERD)
 - Tecnologias utilizadas
 - Decisões técnicas (com trade-offs)
 - Pressupostos assumidos (ver secção 10)
 - Capturas de ecrã da aplicação
 - Divisão de trabalho dentro do grupo

10. Pressupostos Técnicos Assumidos

Estes pressupostos preenchem lacunas do enunciado e devem ser **documentados no relatório final**.

#	Lacuna do enunciado	Decisão adotada	Justificação
1	"Recuperação da senha" — método não especificado	Token único + expiração (link por e-mail simulado ou apresentado no Swagger em dev)	Padrão da indústria; seguro e reversível
2	"Administrador do sistema" — como é criado?	Seed na BD ao arranque (<code>IsAdmin = true</code>)	Controlado, sem rota pública de promoção
3	Perfil privado "acessível apenas a utilizadores autorizados" — quem são?	Dono + seguidores (sem pedido pendente)	Solução simples; reversível com coluna <code>IsAccepted</code> se o professor pedir
4	Notificações "atualização dinâmica" — em tempo real?	Polling a cada 30s (SignalR opcional, se sobrar tempo)	Cumprir o requisito sem complexidade extra
5	Publicação aceita imagem e vídeo simultaneamente?	Permitidos ambos opcionalmente (campos independentes na BD)	Mais flexível; consistente com "imagem(opcional), vídeo(opcional)"
6	Identificadores das entidades	<code>INT IDENTITY</code> (auto-incremento)	Mais legíveis em URLs e mais leves; GUID é alternativa válida se a equipa preferir
7	Localização dos uploads	<code>wwwroot/uploads/{photos,media}/</code> servidos via <code>UseStaticFiles()</code>	Suficiente para projeto académico; sem necessidade de cloud storage
8	Estilização do frontend	TailwindCSS (default) ou Bootstrap 5	Tailwind para responsividade rápida; equipa pode decidir

11. Roadmap de Estudos Avançados (Pós-Parcelar)

Os fundamentos necessários para a 2.^a Parcelar (HTTP, REST, JWT, EF Core, Angular Forms/Router, Reactive Forms, Repository, Service, DTOs, Upload de ficheiros, RxJS básico, TailwindCSS/Bootstrap) estão detalhadamente listados no [Plano da 2.^a Parcelar](#) (secções 9 e 10).

Esta secção cobre apenas os **tópicos avançados** necessários **depois** da parcelar — para fechar tudo o que ficou de fora e levar o projeto a 20/20 no Exame de Época Normal.

11.1. Backend Avançado (pós-parcelar)

Tópico	Onde aplica	Recursos
Constraints UNIQUE compostas no EF Core	Tabela <code>Bazes</code> (PostId + UserId)	YouTube: Milan Jovanović — "EF Core Unique Constraints"
Queries paginadas otimizadas (<code>Skip</code> + <code>Take</code> + <code>Count</code> em paralelo)	<code>FeedController</code> personalizado	YouTube: Nick Chapsas — "Pagination in EF Core"
EF Core projections (<code>.Select(p ⇒ new PostDto { ... })</code>)	Feed sem <code>Include</code> pesado	YouTube: Milan Jovanović — "EF Core Performance: Projections"
AsNoTracking	Queries read-only	Microsoft Learn — <i>Tracking vs No-Tracking Queries</i>
Authorization Policies + Roles	Endpoint admin (moderação)	YouTube: Patrick God — "Role-based Authorization in .NET 8"
Lógica de Notificações com eventos	Disparar notificações em Like/Comment/Follow	YouTube: Milan Jovanović — "Domain Events in .NET"
Soft delete pattern	<code>IsDeleted</code> em Posts/Comments	YouTube: IAmTimCorey — "Soft Delete Pattern"
SignalR (opcional)	Notificações em tempo real	YouTube: Patrick God — "SignalR Tutorial"
Rate limiting	Proteção contra spam de bases/comentários	YouTube: Nick Chapsas — "Rate Limiting in .NET 7+"
Logging estruturado (Serilog)	Diagnóstico em produção	YouTube: Milan Jovanović — "Serilog in .NET"

11.2. Frontend Avançado (pós-parcelar)

Tópico	Onde aplica	Recursos
OnPush Change Detection	Performance em listas longas	YouTube: Joshua Morony — "ChangeDetectionStrategy.OnPush"
Infinite scroll (<code>ngx-infinite-scroll</code>)	Feed personalizado	YouTube: Decoded Frontend — "Infinite Scroll in Angular"
Optimistic UI updates	Toggle de base (incrementa antes da resposta)	YouTube: Joshua Morony — "Optimistic Updates Pattern"
Real-time polling com <code>interval</code> + <code>switchMap</code>	Notificações	YouTube: Decoded Frontend — "Polling with RxJS"
Virtual Scroll (<code>@angular/cdk/scrolling</code>)	Lista grande de comentários	Angular CDK docs
<code>@Input()</code> com setters / Signals (Angular 17+)	Reatividade fina	YouTube: Joshua Morony — "Angular Signals"
<code>async</code> pipe em todo o lado	Evitar memory leaks	YouTube: Decoded Frontend — "Stop subscribing manually"
Lazy load de imagens (<code>loading="lazy"</code>)	Performance no feed	MDN — <i>Loading attribute</i>
Acessibilidade (ARIA, focus management, contraste)	Requisito não-funcional	YouTube: Kevin Powell — "Web Accessibility Tutorial"
i18n (opcional)	App em múltiplos idiomas	Angular docs — <i>Internationalization</i>
PWA (opcional)	Instalável + offline básico	YouTube: Decoded Frontend — "Angular PWA"

11.3. Sequência sugerida pós-parcelar (4 semanas)

Semana 5 – Bases + Feed personalizado

Backend: Constraint UNIQUE + ToggleLike + FeedController paginado

Frontend: like-button (optimistic) + feed-page com infinite scroll

Semana 6 – Follow + Notificações

Backend: FollowsController + NotificationService (eventos)

Frontend: notification-bell + notifications-page + polling

Semana 7 – Privacidade + Moderação

Backend: Lógica de perfis privados + AdminController + Policies

Frontend: UI condicional perfil privado + painel admin

Semana 8 – Polish + Relatório Final

Hardening de segurança · Acessibilidade · Performance · Vídeo demo · Relatório PDF

11.4. Para o Relatório Final (defesa)

Secção do relatório	Conteúdo essencial
Introdução	Contexto, objetivo, público-alvo
Análise de Requisitos	Funcionais + não-funcionais (mapear ao enunciado)
Arquitetura	Diagrama de camadas + justificação de escolhas
Modelo de Dados	ERD + script SQL + normalização (3FN)
Tecnologias	Angular, ASP.NET, SQL Server (com versões)
Decisões Técnicas	JWT vs cookies; AutoMapper vs manual; etc.
Pressupostos Assumidos	Os 8 da secção 10 deste plano
Capturas de Ecrã	Todos os principais fluxos
Divisão do Trabalho	Quem fez o quê + branches
Conclusão	O que aprenderam, dificuldades, próximos passos
Referências	Links dos tutoriais usados, documentação oficial

12. Glossário de Termos Técnicos

Lista de termos usados ao longo do plano. Se vires um termo que não conheces, consulta aqui antes de procurar fora.

Geral / Arquitetura

Termo	Significado
API	Application Programming Interface. Conjunto de endpoints que o backend expõe para o frontend consumir.
REST	Representational State Transfer. Estilo de API que usa URLs + verbos HTTP com JSON.
Endpoint	Um ponto da API. Ex: <code>POST /api/auth/login</code> .
JSON	JavaScript Object Notation. Formato de texto para trocar dados.
CORS	Cross-Origin Resource Sharing. Permite o frontend (<code>:4200</code>) chamar o backend (<code>:5000</code>).
Stack	Conjunto de tecnologias do projeto (Angular + ASP.NET + SQL Server).
Camadas	Divisão por responsabilidade: Controller → Service → Repository → BD.
Monorepo	Um único repositório para backend + frontend.

Frontend (Angular)

Termo	Significado
Component	Bloco reutilizável de UI (HTML + TS + SCSS).
Module	Agrupamento de components/services.
Service (Angular)	Classe injetável com lógica reutilizável não-UI.
Reactive Forms	Formulários com validação programática em TS.
HttpClient	Cliente HTTP do Angular.
Observable / RxJS	Stream de valores ao longo do tempo.
BehaviorSubject	Observable que guarda o último valor — bom para estado partilhado.
Guard	Decide se uma rota pode ser acedida (ex: <code>AuthGuard</code>).
Interceptor	Filtro global de pedidos HTTP (ex: injetar JWT).
Lazy Loading	Carregar um módulo só quando necessário.
<code>@Input()</code> / <code>@Output()</code>	Passar dados pai→filho / emitir eventos filho→pai.
<code>*ngFor</code> / <code>*ngIf</code>	Diretivas estruturais.
Pipe	Transforma valor no template (<code>{{ date timeAgo }}</code>).
Standalone Component	Component sem precisar de <code>NgModule</code> (Angular 17+).
Signal	Estado reativo moderno (Angular 17+).
<code>FormData</code>	Objeto JS para enviar ficheiros (multipart).
localStorage	Storage do browser, persistente.
Optimistic Update	UI atualizada antes da resposta do servidor.
OnPush Change Detection	Estratégia que melhora performance.

Backend (ASP.NET / C#)

Termo	Significado
Controller	Classe C# que define endpoints REST.
Action	Método público de um Controller (= 1 endpoint).
DTO	Data Transfer Object. Classe usada na API (input/output). Não expor entidades de BD.
Entity / Modelo	Classe C# que mapeia para uma tabela.
DbContext	Sessão com a BD no EF Core (<code>DbSet<></code> por tabela).
Migration	Snapshot versionado de mudanças à BD.
EF Core	Entity Framework Core — ORM que traduz C# em SQL.
LINQ	Sintaxe C# de query (<code>Where</code> , <code>Select</code> , <code>Include</code>).
Code-First	Escreves classes; EF gera tabelas.
Repository	Camada que isola o acesso a dados.
Service (Backend)	Camada com a lógica de negócio.
Dependency Injection (DI)	.NET entrega dependências às classes (sem <code>new</code>).
Middleware	Componente do pipeline ASP.NET (auth, CORS, exceptions).
<code>[Authorize]</code>	Atributo que protege um endpoint.
Claims	Pares chave-valor dentro do JWT.
JWT	JSON Web Token. String assinada que prova quem és.
BCrypt	Algoritmo para hash de passwords.
AutoMapper	Mapeia automaticamente Entidade ↔ DTO.
FluentValidation	Validação de DTOs com regras encadeadas.
Swagger / OpenAPI	Documentação interativa da API (<code>/swagger</code>).
<code>IFormFile</code>	Tipo .NET para ficheiros via multipart.
<code>async</code> / <code>await</code>	Código assíncrono não-bloqueante.

Base de Dados (SQL Server)

Termo	Significado
Tabela	Conjunto de linhas com a mesma estrutura.
PK (Primary Key)	Coluna(s) que identificam unicamente cada linha.
FK (Foreign Key)	Coluna que aponta para a PK de outra tabela.
UNIQUE constraint	Garante valores únicos (ex: <code>(PostId, UserId)</code> em <code>Bazes</code>).
CHECK constraint	Validação a nível de BD (ex: <code>FolloweId <> FollowedId</code>).
Índice	Acelera queries em colunas (ex: <code>IX_Posts_CreatedAt</code>).
3FN	Terceira Forma Normal — sem duplicação, dependências apenas da PK.
JOIN	Combina linhas de duas tabelas.
<code>GETUTCDATE()</code>	Data/hora UTC atual no SQL Server.
SSMS	SQL Server Management Studio — UI gráfica.

Git / Colaboração

Termo	Significado
Repo	Repositório Git.
Branch	Ramo paralelo de desenvolvimento.
Commit	Snapshot guardado no histórico.
Push / Pull	Enviar/receber commits do/para o remote.
Merge	Juntar uma branch a outra.
Pull Request (PR)	Pedido formal para juntar uma branch, com revisão.
Conflito	Dois commits alteram a mesma linha — resolve-se manualmente.
Tag	Marca um commit como versão (<code>v0.5-parcelar</code>).

Termo	Significado
Baze	Reação à publicação (equivalente cultural angolano do "like").
Follow	Relação onde um utilizador segue outro.
Feed	Lista cronológica de publicações.
Notification	Aviso gerado por baze, comentário ou novo seguidor.

Porquê inglês no código? Convenção universal — frameworks (EF Core, ASP.NET, Angular) esperam inglês, toda a documentação e Stack Overflow estão em inglês, e termos como `Repository`, `Service`, `Controller`, `Middleware`, `Token` não traduzem bem. Exceção: `Baze` é termo cultural angolano e é mantido. Comentários e mensagens ao utilizador final ficam em português.

Nota final: este plano não inventa funcionalidades fora do enunciado. Toda a estrutura segue **estritamente** os requisitos, com a profundidade técnica necessária para ser implementada e defendida com rigor. *May the code be with you.*